Reproducibility in Science Workshop

Goal: Improve reproducibility, collaboration, and computational confidence

1.0 Command-Line Literacy (Why the Shell Matters)

- Most research computing workflows eventually leave the IDE. Comfort with the shell is a force multiplier.

Core Concepts

- Filesystems are hierarchical, not project-aware
- The shell is a scripting language, not just a launcher
- Most research infrastructure assumes CLI fluency

Essential Commands

Prints the current working directory

`pwd`

Opens the current directory

`explorer .`

List all files in a directory

`ls`

Lists files with details such as size and modification date.

`ls -l`

Changes the current directory

`cd /path/to/directory`

Goes back one directory level

`cd ..`

Creates a new directory

`mkdir new_directory`

Creates multiple directories at once

`mkdir data_raw data_processed outputs`

Removes an empty directory

`rmdir old_folder`

Create a new file

`New-Item newfile.txt`

Move files

`mv newfile.txt /new/destination/folder/destination.txt`

Remove files

`rm newfile.txt`

2.0 Git fundamentals

Git, Source Control, and Project History

- Git is a distributed version control system that records the history of a project as a sequence of commits. Each commit is a snapshot of the project state at a point in time.

its how we iteratively save our changes to our code

- The **main branch** represents the authoritative history of the project. It should always reflect a coherent, working state and typically corresponds to results you trust, share, or publish.

- A **branch** is a separate line of development created from an existing commit. Branches allow you to experiment, refactor, or test ideas without modifying main. This enables safe, parallel development while preserving full history and provenance.

- When work on a branch is complete, it can be merged back into main, explicitly incorporating its changes into the official project record.

In source control terms:

- **Commits** track change
- **Branches** isolate intent
- **Merges** formalize decisions

Visual of a working repo

```
(main)
  |
  A --- B --- C ------------------- D
              \                    /
               E --- F --- G ------
            (feature_branch)

A → B → C → D are commits on main

feature_branch is created at commit C

E → F → G are commits made during experimentation

The merge integrates feature_branch back into main at D
```

Git Basic Controls

Create your own GitHub Repo

1. To make a new repo go to www.github.com and create a new repo:

make repo

2. Set the permissions and settings

settings

3. Setting the branch permissions

branch

4. Clone the repo: `git clone https://github.com/cscarpon/Science_Workshop.git`

clone

5. Change to the newly created directory (dir)

   `cd Science_Workshop`

6. Make a branch

   `git checkout -b 2026-02-11-InitialCommit`

7. make a change (add a folder)

8. `git add -A`

9. `git commit -m "We added the test.py file"`

10. `git push`

11. `git push upstream`

12. On the Github repo, review the changes for the PR

13. Merge.

14. `.gitignore` Inside of this file in the root dir, you can put all the things you do not want to upload to Github such as large datafiles or passwords

```
.Rproj.user
.Rhistory
.RData
.Ruserdata
data/mount_roads10m.geojson
```

Adding large files to Github through Large File Storage (LFS)

Although its not always recommended to add ALL of your data, having a test dataset is usually important for users to run your code. LFS also 1gb of free storage per repo. You can store more data if needed, but you have to pay.

1. `git lfs install`
2. Track specific File Types:

- `git lfs track "*.las"`

- `git lfs track "*.laz"`
- `git lfs track "*.tif"`

3. `git add -A`

4. `git commit -m "Adding data"`

Verifying which files are being tracked by LFS by running:

`git lfs ls-files`

Other handy commands

Initialize a new Git repository in the current directory

`git init`

or we can create a new one on the github website itself.

Check the current status of the repository (Shows tracked, untracked, and modified files)

`git status`

Add a file to the staging area (Staging marks files to be included in the next commit)

`git add analysis.R`

Add all modified and new files to the staging area

`git add -A`

Create a commit (A commit records a snapshot of the project state)

`git commit -m "Add initial analysis script"`

Show the commit history

`git log`

Create a new branch for experimental work without swapping to it.

`git branch feature_experiment`

Switch to the new branch

```
git checkout feature_experiment
```

Shortcut: create and switch to a branch in one command

```
git checkout -b yyyy-mm-dd-feature_experiment
```

```
#git checkout -b 2026-01-16-yml
```

Switch back to the main branch

```
git checkout main
```

Merge changes from the feature branch into main

```
git merge feature_experiment
```

Push to Remote Repository

```
git remote add origin https://github.com/cscarpon/Science_Workshop.git
```

once you have set the remote, you can push changes

```
git push
```

If you are checking for an updated version from the remote repository

```
git checkout main
```

```
git pull
```

3.0 Reproducible Research Foundations

Reproducibility as a Workflow Property

Reproducibility is not achieved at publication time; it is a consequence of how a project is structured and executed. A reproducible project allows someone else, or your future self, to regenerate results using the same code, data, and computational environment.

This section focuses on:

- Structuring projects so intent is clear
- Separating raw data from derived outputs
- Capturing the computational context in which results were produced
- Reproducible workflows reduce ambiguity, prevent accidental data corruption, and make research easier to resume after interruptions.

3.1 Environment Management (Conda)

Isolating Computational Context

An environment defines the exact versions of software and libraries used in a project. Without explicit environment management, analyses become fragile and difficult to reproduce across machines or over time.

Conda environments:

- Isolate dependencies from the system and other projects
- Allow exact recreation of software stacks
- Capture part of the research method, not just implementation details
- The environment specification file should be treated as part of the project's methodology and versioned alongside the code.

Create a new Conda environment named "research_env"

This environment is isolated from the base environment

```
conda create -n research_env python=3.11
```

Activate the Conda environment

```
conda activate research_env
```

Install common scientific Python packages

```
conda install numpy pandas matplotlib
```

Export the full environment specification

This records exact package versions and sources

```
conda env export > environment.yml
```

Recreate the environment later or on another machine

```
conda env create -f environment.yml
```

Remove an environment when it is no longer needed

```
conda remove -n research_env --all
```

3.2 R Projects and Research-Oriented Structure

R Projects as Analytical Units

- An R project defines a self-contained analytical workspace. It establishes a fixed project root, making all file paths relative, portable, and reproducible. This avoids hard-coded absolute paths and reduces ambiguity when moving between machines or collaborators.
- An R project is not just an IDE convenience. It is a logical boundary for a research task, experiment, or manuscript. Treating each project as an independent unit improves clarity, reproducibility, and version control.
- R projects work naturally with Git, renv, and standardized directory layouts.

Creating an R Project

In RStudio:

```
File → New Project → New Directory → New Project
```

Project Structure and source files

```
project_name/
  project_name.Rproj
```

Recommend Project Structure

```
project_name/
├── data/
│   ├── raw/
│   └── processed/
├── scripts/
├── outputs/
│   ├── figures/
│   └── tables/
├── tests/
│   └── firsttest.R/
├── renv/
├── renv.lock
├── README.md
└── project_name.Rproj
```

Directory Roles

`/data/raw`

- Original input data
- Read-only
- Never overwritten or modified

`/data/processed`

- Cleaned or derived datasets
- Fully reproducible from scripts
- Can be regenerated at any time

`/scripts`

- All analysis and processing code
- No manual data manipulation
- Scripts should be executable end-to-end

`/outputs`

- Figures, tables, model results

- Generated programmatically
- Safe to delete and recreate

`renv/ and renv.lock`

- Captures the R package environment
- Part of the research method

`README.md`

- Explains purpose, structure, and workflow
- Entry point for collaborators and reviewers

3.3 Environment Management with renv in R

renv is a package manager for R much similar to conda for Python. It helps to keep your package versions locked and organised. When using different version of r and rtools (4.X.X), packages will need to be updated or locked depending on the version of R you are using.

Installing renv

`install.packages("renv)`

Initialize a reproducible R environment

`renv::init()`

installing a package

`renv::install("dplyr")`

Record package versions used by the project

`renv::snapshot()`

Restore the exact package environment later

`renv::restore()`

4.0 Package Syntax and Management:

Package management through `roxygen2` or `docstrings` keeps your code clean, professional, and understandable. By using these standards, you ensure a consistent format for help files—accessible via `?function()` in R or `help()` in Python. This allows other researchers to use your tools without needing to read the raw source code.

Upholding these documentation standards is a prerequisite for sharing your work globally. Once your code is properly documented, it is much easier to submit your project to official repositories like CRAN for R or PyPI (the Python Package Index) for Python.

The Package Management Synntax

`roxygen2` : An R package that converts special #' comments into formal .Rd help files and manages the NAMESPACE to define which functions are public. It ensures your R package meets the strict documentation requirements for CRAN.

Key Roxygen2 Tags:

- `@title` : The official name of the function or class.
- `@param` : Defines an input argument, such as a file path or a numeric threshold.
- `@return` : Describes what the function provides back to the user, such as a dataframe or a plot.
- `@export` : This is a critical tag. It tells R to make the function available to the user when they load your package. Without it, the function remains internal and hidden.

Example:

```
#' Filter LiDAR Points by Classification
#'
#' @param las_path String. The file path to the .las or .laz file.
#' @param class_num Numeric. The LAS classification code to extract (e.g.,
2 for Ground, 3 for Low Vegetation).
#' @return An LAS object containing only the points matching the input
classification.
#' @export
filter_las_class <- function(las_path, class_num) {
    #Load the LiDAR file
    las <- lidR::readLAS(las_path)
    filtered_las <- lidR::filter_poi(las, Classification == class_num)
```

```
    return(filtered_las)
}
```

`Docstrings` : Built-in Python strings located inside functions that provide immediate guidance to users. When written in a standard style (like Google or NumPy), they allow tools to automatically generate professional websites and PyPI-ready documentation.

Key Docstring Sections:

- `Args` : The equivalent to @param in R.
- `Returns` : The equivalent to @return in R.
- `Attributes` : This section is used specifically inside classes to describe the variables stored in the object.

```
import laspy
import numpy as np

def filter_las_class(las_path, class_num):
    """
    Filters a LAS file to keep only points of a specific classification.

    Args:
        las_path (str): The file path to the .las or .laz file.
        class_num (int): The integer code for the desired classification.

    Returns:
        laspy.LasData: A new LAS object containing only the filtered
points.
    """
    # Load the LiDAR file
    las = laspy.read(las_path)

    # Create a mask where the classification matches the input
    mask = las.classification == class_num

    # Apply the mask to create a new filtered LAS object
    filtered_las = las[mask]

    return filtered_las
```

5.0 Testing and Validation

Making Assumptions Explicit

Testing in research code is not about achieving full test coverage. It is about making assumptions explicit and checking them automatically.

Tests help ensure that:

- Inputs conform to expected ranges or structures
- Outputs remain scientifically plausible
- Changes to code do not silently alter results
- Lightweight testing reduces the risk of unnoticed errors and increases confidence in analytical pipelines, especially as projects grow or are revisited.
- Usually you have a few different types of tests such as ensuring the conda environment is running, testings that simple python logic works, then testing smaller to larger functions. Often these larger tests would be a full run through of your sample code logic.

5.1 Testing with PyTest

`pytest` is a testing framework that automatically detects and runs your code's validation checks to ensure your scientific models and data processing remain accurate and reproducible. To be recognized by the framework, your test files must be named using the `test_*.py` or `*_test.py` format, and the functions inside must also start with the `test_` prefix.

```
# Install pytest into the active Python environment
%pip install pytest

import pytest

# Create a test file (must start with test_)
# File: test_metrics.py

def test_positive_values():
    values = [1, 2, 3]
    assert all(v > 0 for v in values)

# Run all tests in the project
pytest

# Run tests with verbose output
pytest -v

# Example of validating scientific assumptions
```

```python
def test_no_negative_heights():
    heights = [10.2, 5.4, 3.1]
    assert min(heights) >= 0
```

The `assert` statement is the heart of pytest. It is a simple logical check:

If the expression following it is True, the test passes and moves on.

If the expression is False, it "raises" an error, and pytest marks that specific test as FAILED.

In [10]:
```python
# To run all tests in the repository:
# pytest
import pytest
# To see a detailed breakdown of each test:
!pytest -v ..
```

```
============================= test session starts ==============================
platform win32 -- Python 3.11.14, pytest-8.4.2, pluggy-1.5.0 -- C:\Users\cscar\anaco
nda3\envs\research_env\python.exe
cachedir: .pytest_cache
rootdir: c:\Users\cscar\Science_Workshop
plugins: anyio-4.13.0
collecting ... collected 3 items

..\tests\test_metrics.py::test_positive_values PASSED                    [ 33%]
..\tests\test_metrics.py::test_no_negative_heights PASSED                [ 66%]
..\tests\test_metrics.py::test_load_roads_geojson PASSED                 [100%]

============================== 3 passed in 1.50s ===============================
```

5.2 Testing and Validation in R (testthis)

```r
library(testthat)

test_that("addition works correctly", {
  result <- 2 + 3
  expect_equal(result, 5)
})
```

6.0 Loop-Based Scripts vs Structured Code

Why Loops Limit Reproducibility

Many research scripts rely on hard-coded loops over files and folders. While this works initially, it tightly couples the analysis logic to a specific directory layout, file naming scheme,

and execution context.

This creates several problems:

- The code only works for one exact folder structure
- Re-running the analysis requires manual edits
- Sharing the code requires others to mirror your filesystem
- Provenance is implicit rather than explicit
- In practice, the loop becomes the method, rather than the computation itself.

Typical code presentation for a loop that runs a very specific set of commands, this is hard to apply to different use case scenarios

```
import os

data_dir = os.path.join("..", "data")
out_dir = os.path.join("..", "outputs")

for fname in os.listdir(data_dir):
    if fname.endswith(".csv"):
        in_file = os.path.join(data_dir, fname)
        out_file = os.path.join(out_dir, fname.replace(".csv",
"_processed.csv"))

        with open(in_file) as f:
            data = f.read()

        processed = data.upper()

        with open(out_file, "w") as f:
            f.write(processed)
```

In [5]:
```
import os

data_dir = os.path.join("..", "data")
out_dir = os.path.join("..", "outputs")

for fname in os.listdir(data_dir):
    if fname.endswith(".csv"):
        in_file = os.path.join(data_dir, fname)
        out_file = os.path.join(out_dir, fname.replace(".csv", "_processed.csv"))

        with open(in_file) as f:
            data = f.read()
```

```
        processed = data.upper()

        with open(out_file, "w") as f:
            f.write(processed)
```

Issues:

- Paths are embedded in the logic
- Cannot easily reuse on another dataset
- No record of configuration or intent
- Difficult to test or extend

6.1 Creating an object to store all of your files and functions

- `init` is to ensure that the object is created when loaded
- `self.` is to signify what lives inside the `slots` of the object
- `def function()` are functions that can be called inside the file and work with the stored files in the slots

```
from pathlib import Path

class DataProcessor:
    """
    A tool to batch process CSV files by converting content to uppercase.

    Attributes:
        input_dir (Path): Source directory containing raw .csv files.
        output_dir (Path): Target directory for processed results.
    """

    def __init__(self, input_dir, output_dir):
        """
        The Constructor: Initializes the paths and ensures output folder
exists.
        """
        self.input_dir = Path(input_dir)
        self.output_dir = Path(output_dir)

        # Action: Create the folder if it doesn't exist
        self.output_dir.mkdir(parents=True, exist_ok=True)

    def process_file(self, file_path):
        """
```

```
        Logic for a single file. Isolating this makes it easier to test!

        Args:
            file_path (Path): The path to the file to be processed.
        Returns:
            str: The processed data.
        """
        # .read_text() is a modern 'pathlib' way to open and read in one
go
        data = file_path.read_text()
        return data.upper()

    def run(self):
        """
        The main 'Engine': Orchestrates the batch processing across all
files.
        """
        # .glob("*.csv") finds all matching files in the folder
        for path in self.input_dir.glob("*.csv"):
            result = self.process_file(path)

            # Use / operator (pathlib magic) to join paths safely
            # .stem gets the filename without the .csv extension
            out_path = self.output_dir / f"{path.stem}_processed.csv"

            out_path.write_text(result)
            print(f"Done: {path.name} -> {out_path.name}")
```

In [ ]:

```
from pathlib import Path
import os

class DataProcessor:
    """
    A tool to batch process CSV files by converting content to uppercase.

    This class provides a structured way to handle directory-level file
    transformations, ensuring that the output folder is prepared and
    files are handled using modern Pathlib logic.

    Attributes:
        input_dir (Path): Source directory containing the raw .csv files.
        output_dir (Path): Target directory where processed results are saved.
    """

def __init__(self, input_dir, output_dir):
    """
    Initializes the DataProcessor with input and output paths.

    Args:
        input_dir (str or Path): The directory containing files to process.
        output_dir (str or Path): The directory where output files will be written.
    """
    self.input_dir = Path(input_dir)
    self.output_dir = Path(output_dir)
```

```python
    # Action: Create the folder if it doesn't exist
    self.output_dir.mkdir(parents=True, exist_ok=True)

def process_file(self, file_path):
    """
    Reads a single file and converts its text content to uppercase.

    Args:
        file_path (Path): The Path object representing the file to be read.

    Returns:
        str: The entire content of the file converted to uppercase strings.
    """
    # .read_text() is a modern 'pathlib' way to open and read in one go
    data = file_path.read_text()
    return data.upper()

def run(self):
    """
    Orchestrates the batch processing of all CSV files in the input directory.

    This method identifies all files ending in .csv, applies the transformation
    logic via process_file(), and writes the new content to the output directory
    with a '_processed' suffix.

    Returns:
        None
    """
    # .glob("*.csv") finds all matching files in the folder
    for path in self.input_dir.glob("*.csv"):
        result = self.process_file(path)

        # Use / operator (pathlib magic) to join paths safely
        # .stem gets the filename without the .csv extension
        out_path = self.output_dir / f"{path.stem}_processed.csv"

        out_path.write_text(result)
        print(f"Done: {path.name} -> {out_path.name}")
```

6.2 How to execute this class object called `DataProcessor`

```python
data_dir = os.path.join("..", "data")
out_dir = os.path.join("..", "outputs")

processor = DataProcessor(
    input_dir= data_dir,
    output_dir= out_dir
)
```

```
processor.run()
```

In [9]:
```
data_dir = os.path.join("..", "data")
out_dir = os.path.join("..", "outputs")

processor = DataProcessor(
    input_dir= data_dir,
    output_dir= out_dir
)

processor.run()
```

In [ ]:
```
# Diagnostics: See the assigned paths
print(vars(processor))
# Output: {'input_dir': PosixPath('data'), 'output_dir': PosixPath('outputs')}

# Diagnostics: See all available methods and attributes
print(dir(processor))

help(processor)
```

Running this in R

```
library(R6)
library(fs) # The 'pathlib' of R

DataProcessor <- R6Class("DataProcessor",
  public = list(
    input_dir = NULL,
    output_dir = NULL,

    #' @description
    #' The Constructor: Initializes paths and creates the output folder.
    initialize = function(input_dir, output_dir) {
      # Use fs::path() for cross-platform safety (like Path())
      self$input_dir <- path(input_dir)
      self$output_dir <- path(output_dir)

      # Action: Create the folder if it doesn't exist
      dir_create(self$output_dir)
    },

    #' @description
    #' Logic for a single file.
    process_file = function(file_path) {
      data <- read_file(file_path)
      return(toupper(data)) # toupper() is R's .upper()
    },
```

```
    #' @description
    #' The main 'Engine': Orchestrates the batch processing.
    run = function() {
      # dir_ls with a glob pattern finds the files
      files <- dir_ls(self$input_dir, glob = "*.csv")

      for (f in files) {
        result <- self$process_file(f)

        # path_file/path_ext_remove are like .name and .stem in pathlib
        file_stem <- path_ext_remove(path_file(f))
        out_path <- path(self$output_dir, paste0(file_stem,
"_processed.csv"))

        write_file(result, out_path)
        message(paste("Done:", path_file(f), "->", path_file(out_path)))
      }
    }
  )
)

# --- How to use it ---
# processor <- DataProcessor$new(input_dir = "data", output_dir =
"outputs")
# processor$run()
```

6.3 Advantages:

- Directory structure is configurable, not hard-coded
- Logic is isolated and reusable
- Inputs and outputs are explicit
- Easier to test, document, and version

7.0 Documentation and Knowledge Transfer

Writing for Your Future Self

- Documentation is a form of communication between collaborators separated by time. In most projects, your future self is the primary audience.

Clear documentation:

- Explains what the project does and why
- Describes how to run analyses end-to-end
- Records assumptions, decisions, and limitations
- A well-maintained README and clean project structure dramatically reduce the cost of onboarding new collaborators or returning to old work.

Markdown Basics (For Contributors)

To edit this file, use these simple rules:

(#) for Titles, (##) for Sections.

(**Bold**) for emphasis.

(``) `code` for inline commands .

Use triple backticks (```) for code blocks.

Adding (---) puts a long line to separate sections